

# Backend CIP

## Central de Inteligencia e Performance

Resumo do backend desenvolvido para o Grupo MKR, incluindo arquitetura, integrações, sincronização de dados, segurança, rotas entregues e próximos passos.

|         |                                                                                                            |
|---------|------------------------------------------------------------------------------------------------------------|
| Projeto | CIP - Dashboard executivo integrado ao Sankhya                                                             |
| Escopo  | Backend Node.js, API REST, snapshot SQLite, sincronizadores Sankhya, autenticação e indicadores de negocio |
| Data    | 29 de junho de 2026                                                                                        |
| Status  | Implementado na fatia atual, com pendências mapeadas para evolução                                         |

O backend foi construído para centralizar a comunicação com o Sankhya, proteger credenciais, acelerar as consultas do dashboard por meio de um snapshot SQLite e entregar uma camada REST estável para o frontend. A solução já cobre autenticação, sincronização das principais entidades, indicadores financeiros, vendas, estoque, clientes, RH, entregas e integração Via Certa.

|                                             |                                                  |                                                 |                                          |
|---------------------------------------------|--------------------------------------------------|-------------------------------------------------|------------------------------------------|
| <b>Node 22</b><br>Runtime mínimo do backend | <b>SQLite</b><br>Snapshot local para performance | <b>5 min</b><br>Intervalo padrão de sync quente | <b>TOTP</b><br>Autenticação do dashboard |
|---------------------------------------------|--------------------------------------------------|-------------------------------------------------|------------------------------------------|

# 1. Sumario executivo

O backend do CIP e uma API Express em TypeScript responsavel por integrar o dashboard do Grupo MKR ao Sankhya ERP e a servicos externos. Ele executa a coleta de dados, normaliza informacoes de negocio, grava um snapshot local em SQLite e disponibiliza endpoints REST para consumo do frontend.

### O que ja foi entregue

- API REST estruturada em /api.
- Autenticacao por TOTP e token de sessao assinado.
- Cliente Sankhya com OAuth, cache de token e retry.
- Snapshot SQLite com schema versionado.
- Scheduler automatico de sincronizacao.
- Dashboards de vendas, financeiro, estoque, clientes, RH e entregas.
- Integracao Via Certa para alunos ativos.

### Valor pratico

- Reduz dependencia de consultas diretas ao Sankhya em tempo real.
- Evita exposicao de credenciais no navegador.
- Melhora tempo de resposta das telas do dashboard.
- Cria base local para analises, rankings e indicadores consolidados.
- Permite evoluir o produto por modulos sem reescrever a integracao.

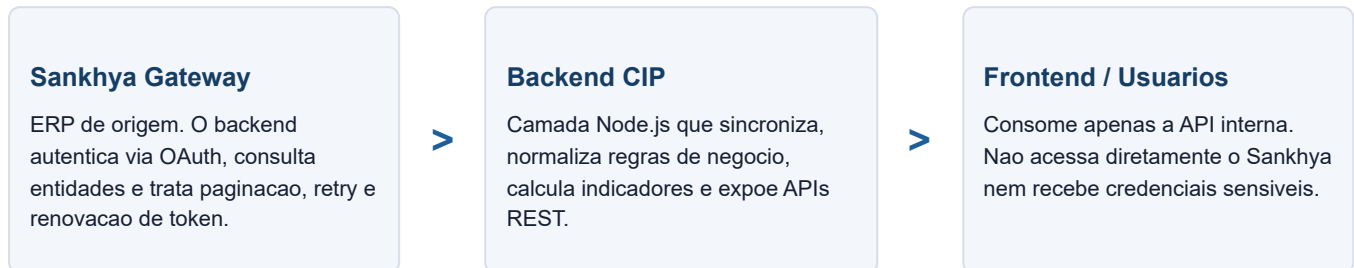
## Tecnologias utilizadas

| Componente              | Uso no backend                                                  |
|-------------------------|-----------------------------------------------------------------|
| Node.js 22 + TypeScript | Base da API e dos sincronizadores.                              |
| Express 5               | Servidor HTTP, middlewares, rotas e tratamento de erros.        |
| better-sqlite3          | Banco local usado como snapshot operacional.                    |
| Zod                     | Validacao de ambiente e parametros de rotas.                    |
| Pino                    | Logs estruturados da API e dos ciclos de sincronizacao.         |
| Sankhya Gateway         | Origem principal dos dados ERP via OAuth e CRUDServiceProvider. |

Este documento e uma versao executiva. A documentacao tecnica detalhada fica em doc/BACKEND\_COMPLETO.md.

## 2. Arquitetura da solucao

A arquitetura foi desenhada para separar tres responsabilidades: integracao com sistemas externos, persistencia de snapshot e entrega de dados prontos para as telas do dashboard.



### Componentes internos

| Camada       | Arquivos principais                            | Responsabilidade                                                                                |
|--------------|------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Bootstrap    | server.ts, config.ts                           | Inicializacao da API, CORS, cache headers, logger, migrations, scheduler e middlewares globais. |
| Autenticacao | auth.ts                                        | TOTP, criacao de token de sessao, validacao Bearer e protecao das rotas.                        |
| Banco        | db/connection.ts, db/migrate.ts, db/schema.sql | Conexao SQLite, schema, indices, migracoes incrementais e metadados.                            |
| Sankhya      | sankhya/client.ts, crud.ts, decoder.ts         | OAuth, requisicoes ao gateway, CRUDServiceProvider, paginacao e decoder de respostas.           |
| Sync         | sync/*.ts                                      | Sincronizacao das entidades e controle de status em sync_state.                                 |
| Services     | services/*.ts                                  | Consultas SQL, regras de negocio e montagem dos DTOs consumidos pelo frontend.                  |
| Rotas        | routes/index.ts, routes/dashboard.ts           | Contratos REST, validacao dos parametros e resposta JSON.                                       |

### Decisoes de arquitetura

- **Snapshot local:** evita consultas pesadas no Sankhya para cada acesso ao dashboard.
- **Backend como unica ponte:** o navegador nao recebe credenciais nem conhece a API Sankhya diretamente.
- **Services separados das rotas:** facilita evoluir regras de negocio sem misturar com HTTP.
- **Sync state por entidade:** permite monitorar frescor, erros e volume sincronizado.

### 3. Dados e sincronizacao

O backend sincroniza dados do Sankhya e grava o resultado em SQLite. A partir desse snapshot, os services calculam os indicadores usados pelo dashboard.

#### Entidades sincronizadas

| Sync           | Origem        | Destino         | Observacao                                                 |
|----------------|---------------|-----------------|------------------------------------------------------------|
| empresas       | Seed local    | empresas        | Usado porque a entidade Empresa esta bloqueada no Sankhya. |
| tipos_operacao | TipoOperacao  | tipos_operacao  | Base para classificar movimentos e faturamento.            |
| naturezas      | Natureza      | naturezas       | Plano de contas usado na DRE.                              |
| tipos_titulo   | TipoTitulo    | tipos_titulo    | Descricao de titulos financeiros.                          |
| parceiros      | Parceiro      | parceiros       | Clientes e fornecedores ativos.                            |
| vendedores     | Vendedor      | vendedores      | Usado em filtros, rankings e BI de RH.                     |
| produtos       | Produto       | produtos        | Cadastro base para produtos e estoque.                     |
| pedidos        | CabecalhoNota | pedidos         | Notas liberadas desde 01/01/2025.                          |
| titulos        | Financeiro    | titulos         | Titulos desde 01/01/2026.                                  |
| estoque        | Estoque       | produto_estoque | Recarga completa do estoque.                               |

#### Agenda de sincronizacao

##### Sync inicial

Ao subir, o backend prepara o banco, executa empresas e tenta carregar dimensoes e fatos ausentes.

- Empresas sempre rodam no inicio.
- Dimensoes rodam se nao houver snapshot.
- Pedidos, titulos e estoque rodam se faltarem dados.

##### Sync periodico

O scheduler separa cargas frequentes e cargas mais lentas.

- **Hot:** pedidos e titulos a cada 5 minutos.
- **Slow:** dimensoes a cada 30 minutos.
- **Manual:** estoque pode ser atualizado via script.

**Ponto de atencao:** produtos e estoque nao estao no ciclo periodico completo no codigo atual. O estoque possui script manual e participa do sync inicial quando o snapshot esta ausente.

## 4. Regras de negocio implementadas

### Faturamento real

O backend nao considera todo movimento de venda como faturamento. Foi criada uma whitelist de TOPs para evitar inflar os indicadores com remessas, ajustes, bonificacoes ou comodato.

**Regra aplicada:**

```
CODTIPOPER IN (FATURAMENTO_TOPS) AND STATUSNOTA = 'L' AND DTFATUR IS NOT NULL
```

TOPs mapeados: 1100, 1107, 1111, 1716, 1733, 1763, 1776, 1795, 1797, 1801, 1802, 1705, 1766, 1769, 1770.

### Comodato

Comodato foi separado do faturamento e possui indicador proprio de saidas, retornos e saldo ativo.

| Categoria           | TOPs                                             |
|---------------------|--------------------------------------------------|
| Saida de comodato   | 1109, 1772                                       |
| Retorno de comodato | 1203                                             |
| Saldo ativo         | Historico de saidas menos historico de retornos. |

### DRE e financeiro

A DRE simplificada combina a receita calculada por notas faturadas com despesas extraidas dos titulos financeiros, classificadas pelo prefixo da natureza.

| Prefixo CODNAT | Categoria                            | Entra no total operacional?            |
|----------------|--------------------------------------|----------------------------------------|
| 1              | Receitas                             | Nao, compoe receita bruta              |
| 2              | Custos / Estoques                    | Sim                                    |
| 3              | Despesas Administrativas             | Sim                                    |
| 4              | Despesas Comerciais                  | Sim                                    |
| 5              | Impostos / Tributos                  | Sim                                    |
| 6, 7, 8        | Investimentos, dividendos e servicos | Nao entram no operacional simplificado |

### Filtros padrao

- **Empresa:** aceita todas, um codigo ou lista separada por virgula.
- **Vendedor:** aceita todos, um codigo ou lista separada por virgula.
- **Datas:** rotas com data usam formato YYYY-MM-DD.
- **Paginas:** rotas paginadas usam page iniciado em zero e pageSize limitado.

## 5. APIs e modulos entregues

Todas as rotas ficam sob o prefixo /api. As rotas publicas sao apenas health e autenticacao inicial; as demais exigem token Bearer de sessao.

### Rotas publicas e autenticacao

| Rota                    | Uso                                                |
|-------------------------|----------------------------------------------------|
| GET /api/health         | Status da API, banco, sync e contagens principais. |
| GET /api/auth/setup     | Dados para cadastrar autenticador TOTP.            |
| POST /api/auth/validate | Valida codigo TOTP e gera token de sessao.         |
| GET /api/auth/session   | Confere se a sessao atual continua valida.         |

### Rotas de dashboard

| Grupo                   | Principais rotas                                                                      |
|-------------------------|---------------------------------------------------------------------------------------|
| Cadastros               | /api/empresas, /api/vendedores                                                        |
| Empresas e vendas       | /api/dashboard/empresa/faturamento, /faturamento-por-empresa, /resumo, /comodato      |
| Vendedores              | /api/dashboard/vendedores/ranking, /hoje                                              |
| Financeiro              | /api/dashboard/financeiro/dre, /resumo, /fluxo-caixa, /distribuicao-despesas, /contas |
| Estoque                 | /api/dashboard/estoque, /api/dashboard/produtos                                       |
| Clientes, RH e entregas | /api/dashboard/clientes, /rh, /entregas                                               |
| Via Certa               | /api/viacerta/alunos-ativos                                                           |
| Compatibilidade         | /api/receber, /api/pagar                                                              |

### Principais respostas de negocio

- **Faturamento:** dia, semana de 7 dias, mes atual, ano atual e distribuicao por empresa.
- **Financeiro:** DRE, fluxo de caixa, despesas por categoria e contas abertas.
- **Estoque:** saldos, alertas abaixo do minimo, saldos negativos e agrupamento por local.
- **Clientes:** compradores do ano, receita, ticket medio, receber aberto e top clientes.
- **RH:** vendedores ativos, ranking, faturamento por vendedor e evolucao mensal.
- **Entregas:** SLA de 3 dias, atrasos, transportadoras, frete, volumes e entregas recentes.

## 6. Seguranca, configuracao e operacao

### Seguranca

#### Protecao do dashboard

- TOTP baseado em segredo Base32.
- Token de sessao assinado com HMAC-SHA256.
- Expiracao padrao de 8 horas.
- Rotas de negocio exigem Bearer token.

#### Protecao da integracao

- Credenciais Sankhya ficam no ambiente do backend.
- Frontend nunca acessa Sankhya diretamente.
- CORS e configurado por origem permitida.
- Erros sao tratados por middleware centralizado.

### Variaveis principais

| Variavel                           | Finalidade                               |
|------------------------------------|------------------------------------------|
| SANKHYA_BASE_URL                   | URL base do Gateway Sankhya.             |
| SANKHYA_CLIENT_ID / SECRET / TOKEN | Credenciais da aplicacao no Sankhya.     |
| APP_TOTP_SECRET                    | Segredo do autenticador.                 |
| APP_SESSION_SECRET                 | Chave de assinatura das sessoes.         |
| DATABASE_PATH                      | Caminho do snapshot SQLite.              |
| CORS_ORIGINS                       | Lista de origens do frontend permitidas. |
| SYNC_ENABLED                       | Liga ou desliga o scheduler automatico.  |

### Deploy

O backend esta preparado para Railway via backend/railway.json.

- Build: `npm install && npm run build`.
- Start: `npm run start`.
- Healthcheck: `/api/health`.
- Politica de restart: `ON_FAILURE`.

**Atencao em producao:** o SQLite precisa de armazenamento persistente caso o snapshot deva sobreviver a deploys e reinicios. Sem volume persistente, o banco sera recriado e dependera do sync inicial.

## 7. Pendencias conhecidas e recomendacoes

### Pendencias tecnicas ja mapeadas

| Item                                     | Impacto                                                                                             | Prioridade sugerida |
|------------------------------------------|-----------------------------------------------------------------------------------------------------|---------------------|
| Sync de itens de pedido                  | A tabela existe, mas ainda nao ha sincronizador ativo. Limita analises detalhadas por item/produto. | Media               |
| Estoque fora do ciclo periodico completo | Hoje depende de sync inicial ou script manual para atualizacao recorrente.                          | Media               |
| Empresa bloqueada no Sankhya             | Uso de seed local e stubs. Ideal liberar permissao da entidade Empresa.                             | Media               |
| Whitelist manual de TOPs                 | Novos tipos de operacao de faturamento precisam ser incluidos manualmente.                          | Alta                |
| Validar data real de caixa               | Confirmar se DHBAIXA e suficiente ou se deve usar DHCONCIL/DTCONTAB.                                | Alta                |
| Testes automatizados                     | Reduz risco ao alterar TOPs, SQLs de indicadores e decoder Sankhya.                                 | Alta                |
| Rota administrativa de refresh           | Permitiria forcar sincronizacao sem reiniciar servidor ou aguardar intervalo.                       | Baixa               |

### Recomendacoes para a proxima etapa

1. **Confirmar regras financeiras com o time responsavel:** principalmente TOPs de faturamento e data correta de caixa realizado.
2. **Definir estrategia de persistencia do SQLite em producao:** volume no Railway ou migracao futura para Postgres.
3. **Colocar estoque no scheduler periodico:** se a tela de estoque for operacional e precisar de dados sempre frescos.
4. **Criar testes para regras criticas:** decoder do Sankhya, filtros de faturamento, DRE e contas abertas.
5. **Formalizar monitoramento:** acompanhar tempo de sync, quantidade de linhas, erros por entidade e atraso do snapshot.

### Resumo final

O backend ja possui uma base solida para operar o dashboard: integra com o Sankhya, guarda snapshot local, expoe APIs organizadas, protege as rotas com autenticao propria e consolida indicadores importantes para a gestao. As pendencias restantes sao evolucoes normais de maturidade: testes, observabilidade, refinamento de regras financeiras e ampliacao de alguns sincronizadores.